

Ways to Increase LCS by One - Complete Notes

Problem Statement

Given two strings:

```
s1 (length = n1)
s2 (length = n2)
```

Insert **exactly one character** into **s1** such that:

```
new_LCS = old_LCS + 1
```

Return the number of ways to do this.

A way is defined by:

1. Position of insertion.
2. Character inserted.

Example

```
s1 = "ab"
s2 = "acb"
```

Current LCS:

```
LCS = 2
```

Insert:

```
a | c | b
```

New string:

```
acb
```

New LCS:

```
3
```

Answer:

```
1
```

What are they asking?

They are NOT asking:

```
Find new LCS.
```

They are asking:

```
How many insertions can increase the LCS by exactly one?
```

Brute Force

For every position:

```
0 to n1
```

For every character:

```
'a' to 'z'
```

Insert it.

Recompute LCS.

If:

```
new_lcs == old_lcs + 1
```

increment answer.

Complexity

Insertion positions:

```
O(n1)
```

Characters:

```
O(26)
```

LCS:

```
O(n1*n2)
```

Total:

```
O(26*n1*n1*n2)
```

Too large.

Important Observation

The inserted character MUST belong to `s2`.

Why?

Suppose:

```
s2 = "abc"
```

Insert:

```
'z'
```

Can 'z' contribute to LCS?

No.

Because LCS requires matching characters.

Therefore:

```
Only characters present in s2 matter.
```

Main Idea

Suppose we insert a character at position:

```
i
```

The string becomes:

```
LEFT  
INSERTED CHARACTER  
RIGHT
```

Example:

```
s1 = "ab"
```

Insert 'c' at:

```
i = 1
```

Then:

a | c | b

Huge Observation

The inserted character becomes part of the new LCS.

Therefore:

```
LCS on left  
+  
inserted character  
+  
LCS on right
```

So:

```
new_lcs =  
left_lcs + 1 + right_lcs
```

This is the entire solution.

Example

```
s1 = "ab"  
s2 = "acb"
```

Insert:

a | c | b

Left Side

a

a

LCS:

1

Inserted Character

c
c

Contribution:

1

Right Side

b
b

LCS:

1

Total:

$1 + 1 + 1 = 3$

which is:

$\text{old_lcs} + 1$

Why do we need Prefix and Suffix LCS?

We need:

```
left_lcs  
right_lcs
```

quickly.

Doing LCS every time is too expensive.

So we precompute.

Prefix Table

Definition:

```
pref[i][j]  
=  
LCS(  
    s1[:i],  
    s2[:j]  
)
```

Meaning:

```
First i characters of s1  
First j characters of s2
```

This is exactly the normal LCS table.

Example

```
s1 = "ab"  
s2 = "acb"
```

pref[1][1]

a
a

LCS:

1

pref[2][3]

ab
acb

LCS:

2

Building Prefix Table

```
if s1[i-1] == s2[j-1]:
    pref[i][j] = 1 + pref[i-1][j-1]
else:
    pref[i][j] = max(
        pref[i-1][j],
        pref[i][j-1]
    )
```

Exactly normal LCS.

Suffix Table

Definition:

```
suff[i][j]
=
```



```
LCS(  
    s1[i:],  
    s2[j:]  
)
```

Meaning:

Suffix of s1 starting at i
Suffix of s2 starting at j

Example

```
s1 = "ab"  
s2 = "acb"
```

suff[1][2]

```
s1[1:] = "b"  
s2[2:] = "b"
```

LCS:

1

suff[0][0]

```
ab  
acb
```

LCS:

2

Building Suffix Table

This is just LCS from the back.

```
if s1[i] == s2[j]:
    suff[i][j] = 1 + suff[i+1][j+1]
else:
    suff[i][j] = max(
        suff[i+1][j],
        suff[i][j+1]
    )
```

Why do we need suffix?

Because after matching the inserted character, we still need:

```
best LCS from the remaining right portions.
```

That's exactly:

```
suff[i][j]
```

Why j+1 ?

This is the MOST IMPORTANT PART.

Suppose:

```
s1 : a c b
s2 : a c b
      ↑
      j
```

The inserted 'c' already matched:

```
s2[j]
```

Can we use it again?

NO.

LCS cannot reuse characters.

Therefore, the remaining right side must begin from:

```
j+1
```

Diagram

```
s1 : a | c | b
s2 : a | c | b
      ↑
      j
```

After consuming:

```
c
```

the remaining problem becomes:

```
b
b
```

which is:

```
suff[i][j+1]
```

Final Formula

If we insert a character matching:

```
s2[j]
```

then:

```
left
=
pref[i][j]
```

```
middle
=
1
```

```
right
=
suff[i][j+1]
```

Total:

```
pref[i][j]
+
1
+
suff[i][j+1]
```

If:

```
pref[i][j]
+
1
+
suff[i][j+1]
==
old_lcs + 1
```

then this insertion works.

Duplicate Handling

Suppose:

```
s2 = "aaa"
```

At one insertion position:

```
j=0  
j=1  
j=2
```

may all satisfy the formula.

But inserting:

```
'a'
```

at that position counts only once.

Therefore:

```
used = set()
```

for every insertion position.

Code

```
class Solution:  
    def waysToIncreaseLCS(self, n1, n2, s1, s2):  
  
        # Prefix LCS  
        pref = [[0]*(n2+1)  
                for _ in range(n1+1)]  
  
        for i in range(1,n1+1):  
            for j in range(1,n2+1):  
  
                if s1[i-1] == s2[j-1]:  
                    pref[i][j] = (  
                        1 +  
                        pref[i-1][j-1]  
                    )  
                else:
```

```
        pref[i][j] = max(
            pref[i-1][j],
            pref[i][j-1]
        )

old_lcs = pref[n1][n2]

# Suffix LCS
suff = [[0]*(n2+1)
         for _ in range(n1+1)]

for i in range(n1-1,-1,-1):
    for j in range(n2-1,-1,-1):

        if s1[i] == s2[j]:
            suff[i][j] = (
                1 +
                suff[i+1][j+1]
            )
        else:
            suff[i][j] = max(
                suff[i+1][j],
                suff[i][j+1]
            )

ans = 0

# try every insertion position
for i in range(n1+1):

    used = set()

    # try matching inserted character
    # with every position in s2
    for j in range(n2):

        ch = s2[j]

        if ch in used:
            continue

        left = pref[i][j]

        right = suff[i][j+1]

        if (
            left
            + 1
            + right
            == old_lcs + 1
        ):
            ans += 1
            used.add(ch)
```

```
return ans
```

Complexity

Building Prefix Table:

```
O(n1*n2)
```

Building Suffix Table:

```
O(n1*n2)
```

Checking all insertions:

```
O(n1*n2)
```

Total:

```
O(n1*n2)
```

Space:

```
O(n1*n2)
```

Pattern Learned

This problem introduces a NEW DP pattern:

```
Prefix DP
+
One forced operation
+
Suffix DP
```

Formula:

```
left answer
+
current operation
+
right answer
```

Similar Problems

1. Ways to Increase LCS by One
2. One Edit Distance
3. Split Array DP problems
4. Remove One Character problems
5. Maximum Score After One Operation
6. Longest Common Subsequence after one modification
7. String problems involving:
 - insert one character
 - delete one character
 - replace one character

Real Interview Pattern

Whenever you see:

```
Perform exactly one operation
```

ask yourself:

```
Can I split the answer into:

left contribution
+
current operation
+
right contribution ?
```


If yes, think:

Prefix DP
+
Suffix DP

This problem is one of the best examples of that pattern.